

SUBAYAN GHOSH

•

ABOUT ME

I am Subayan Ghosh, a highly motivated and results-driven Computer Science Engineering student at the National Institute of Technology Durgapur, graduating in 2026 with a strong CGPA of 8.91/10. My academic journey has instilled a robust foundation in core computer science principles, including Data Structures, Algorithms, Object-Oriented Programming, and System Design. This theoretical knowledge is complemented by extensive practical experience in full-stack development and, more significantly, in the burgeoning fields of Artificial Intelligence and Machine Learning.

My passion lies at the intersection of intelligent systems and practical application. I am deeply fascinated by the potential of AI and ML to solve complex real-world problems, from automating tedious tasks to uncovering hidden insights in vast datasets. My expertise spans across various programming paradigms, with a strong command over Python for machine learning and data science, C++ for high-performance computing and competitive programming, and JavaScript for dynamic web applications. I thrive on building end-to-end solutions, leveraging frameworks like TensorFlow for deep learning, and MERN stack for robust web platforms.

As an aspiring AI/ML Engineer, my career goal is to contribute to the development of cutting-edge intelligent systems that drive innovation and create tangible impact. I aim to work on challenging projects that demand creative problem-solving, advanced algorithmic thinking, and a deep understanding of machine learning principles. I am eager to apply my skills in model building, data pipeline automation, and API integration to transform complex data into actionable intelligence.

What sets me apart is my unique blend of a strong academic record, diverse project experience, and a proven track record of leadership and problem-solving. From winning the Smart India Hackathon (AI Track) to leading a university coding club and mentoring peers, I consistently demonstrate the ability to not only grasp complex technical concepts but also to apply them effectively in collaborative and high-pressure environments. My internships at Infosys and TCS Research have provided invaluable industry exposure, solidifying my ability to develop scalable solutions and integrate ML models into production systems.

•

TECHNICAL EXPERTISE

My technical proficiency is built upon a solid foundation of computer science fundamentals and practical application across various domains, with a strong emphasis on AI/ML and full-stack development.

Programming Languages:

- Python:** Expert proficiency in Python, extensively used for machine learning, deep learning (TensorFlow), data analysis (Pandas, NumPy), scripting, and backend web development (Django,

Flask). My experience includes building complex data pipelines, developing AI models, and creating web applications.

- C++:** Strong command of C++ for competitive programming, Data Structures and Algorithms implementation, and performance-critical applications. This forms the backbone of my problem-solving and algorithmic thinking skills.
- JavaScript:** Proficient in JavaScript for both frontend (React, Next.js) and backend (Node.js, Express.js) development, enabling me to build dynamic and interactive full-stack applications.
- SQL:** Experienced in writing complex queries, managing databases, and optimizing database performance across various relational database systems.

Frameworks & Libraries:

- TensorFlow:** Hands-on experience in building, training, and deploying deep learning models for classification, regression, and natural language processing tasks. Used for developing the core AI logic in projects like the "AI Resume & Portfolio Builder."
- React / Next.js:** Proficient in building modern, responsive, and scalable user interfaces. Utilized Next.js for server-side rendering and optimized performance in projects like the "Campus Marketplace."
- Node.js / Express:** Expertise in developing robust, scalable, and high-performance backend APIs and microservices. Used extensively in projects such as the "Real-time Chat App" and during my internship at Infosys.
- Django:** Experience in developing full-stack web applications with Python, leveraging Django's ORM and MVT architecture for rapid development and secure applications.
- Scikit-learn:** Applied for various machine learning tasks including classification, regression, clustering, and dimensionality reduction, particularly during my Machine Learning Internship at TCS Research.

Tools & Platforms:

- Git:** Expert in version control using Git and GitHub for collaborative development, code management, and maintaining project history.
- Docker:** Proficient in containerizing applications and services, ensuring consistent environments across development, testing, and production. Utilized for deploying microservices during my Infosys internship.
- Linux:** Experienced in working with Linux environments for development, server management, and deploying applications.
- AWS (EC2, S3, RDS):** Hands-on experience with Amazon Web Services for cloud deployment, virtual machine management (EC2), object storage (S3), and managed databases (RDS). Certified AWS Cloud Practitioner.
- Postman:** Regularly used for testing and debugging RESTful APIs during development and integration phases.

Databases:

- MongoDB:** Proficient in NoSQL database management, particularly for scalable and flexible data storage in MERN stack applications like the "Real-time Chat App" and "Campus Marketplace."
- MySQL / PostgreSQL:** Experienced with relational database design, querying, and optimization for structured data storage and management.

Core Concepts & Methodologies:

- **Data Structures & Algorithms:** Deep understanding and practical application of fundamental data structures and algorithms, honed through competitive programming and solving 700+ DSA problems. This underpins my ability to write efficient and optimized code.
- **Object-Oriented Programming (OOP):** Strong grasp of OOP principles (encapsulation, inheritance, polymorphism, abstraction) for designing modular, maintainable, and scalable software systems.
- **REST APIs:** Expertise in designing, developing, and consuming RESTful APIs for seamless communication between frontend and backend services, and for integrating machine learning models.
- **System Design:** Fundamental understanding of designing scalable, fault-tolerant, and high-performance distributed systems, including microservices architecture and caching strategies.
-

PROJECT SHOWCASE

AI Resume & Portfolio Builder

- **Technologies Used:** Python, Streamlit, Gemini API, PDFKit

- **Problem Statement:**

The track
frequent
seekers
recruite

- **Solution Approach:**

This pro
languag
system
customi
Streaml
applicat

- **Key Features:**
- **ATS-Optimized Content Generation:** Utilizes the Gemini API to analyze user-provided data (skills, experience, education) and generate resume sections (summary, experience descriptions, bullet points) that are rich in relevant keywords and structured for ATS compatibility, significantly increasing the chances of passing initial screenings.
- **Dynamic Portfolio Section Creation:** Beyond resumes, the tool assists users in structuring and populating their online portfolios. It suggests relevant project descriptions, technical highlights, and impact statements based on the user's project details, ensuring a comprehensive and engaging showcase of work.
- **Interactive User Interface:** Built with Streamlit, the application provides a user-friendly web interface where users can input their information, preview generated content in real-time, and make adjustments, offering a highly interactive and personalized experience.

- **One-Click PDF Export:**** Integrates PDFKit to enable users to download their generated resumes and portfolios as professional, formatted PDF documents, ready for submission to job applications.
- **Implementation Highlights:****
 - **Prompt Engineering for Gemini API:**** A significant challenge involved crafting effective prompts for the Gemini API to ensure accurate, contextually relevant, and ATS-friendly content generation. This required iterative refinement of prompts to guide the LLM's output and handle diverse user inputs gracefully.
 - **Streamlit State Management & Interactivity:**** Managed complex application state within Streamlit to ensure a seamless user experience, allowing users to navigate between sections, edit inputs, and regenerate content without losing progress. Implemented custom components for enhanced UI/UX.
 - **Scalable Content Generation:**** Designed the system to efficiently handle concurrent requests for content generation, ensuring responsiveness even with a growing user base. Optimized API calls and data processing to minimize latency.
 - **Robust PDF Generation:**** Integrated PDFKit to reliably convert dynamically generated HTML content into high-quality PDF documents, addressing challenges related to styling, pagination, and font rendering for various content lengths.
- **Outcomes & Learnings:****

The AI F
their job
project
applicat
Streaml
efficien

•

Real-time Chat App

- **Technologies Used:**** MongoDB, Express.js, React, Node.js (MERN Stack), Socket.io, JWT (JSON Web Tokens)
- **Problem Statement:****
- **Solution Approach:****
- **Key Features:****
 - **Live Messaging & Group Chats:**** Supports instant one-on-one and group messaging, allowing

Modern
facilitate
efficient
ensuring

This pro
Socket.i
with No
databas
an intuit
exchange

users to send and receive messages in real-time. Messages are delivered instantly without requiring page refreshes, providing a seamless communication experience.

- **User Presence & Status:**** Implements real-time tracking of user online/offline status and typing indicators, enhancing the interactive feel of the chat application and informing users about the availability and activity of their contacts.
- **Secure User Authentication:**** Utilizes JSON Web Tokens (JWT) for secure user registration, login, and session management. This ensures that only authenticated users can access chat functionalities and protects user data.
- **Persistent Message History:**** Stores all chat messages in a MongoDB database, allowing users to access their complete conversation history upon logging in, ensuring no data loss and providing continuity.
- **Real-time Notifications:**** Sends instant notifications to users for new messages, friend requests, or other relevant events, even when the chat window is not actively focused, keeping users engaged.

- **Implementation Highlights:****

- **WebSocket Integration with Socket.io:**** Implemented Socket.io on both the client and server sides to establish persistent WebSocket connections. This involved managing various Socket.io events for message sending, receiving, user presence updates, and error handling, ensuring reliable real-time data flow.
- **Scalability for Concurrent Users:**** Designed the backend to handle a large number of concurrent WebSocket connections efficiently. Explored strategies for scaling Socket.io horizontally using Redis adapters for multi-server deployments (though not fully implemented, the design considered it).
- **Secure API Design & Authentication:**** Developed a secure RESTful API using Express.js, protected by JWT authentication. Implemented middleware for token verification and user authorization to secure endpoints and prevent unauthorized access to chat data.
- **MongoDB Schema Design:**** Designed a flexible and efficient MongoDB schema for storing user profiles, chat messages, and conversation threads. Utilized Mongoose for object data modeling, including indexing for faster query performance and relationships between collections.

- **Outcomes & Learnings:****

-

Smart Code Analyzer

- **Technologies Used:**** Python, NLTK, spaCy, Abstract Syntax Trees (AST), Regular Expressions
- **Problem Statement:****

- **Solution Approach:****

This project
significantly
the complex
experiences
systems

Manual
large code
complex
reducing

- Key Features:**

- Automated Bug Detection:** Identifies common programming errors and anti-patterns such as unhandled exceptions, potential null pointer dereferences, incorrect loop conditions, and resource leaks by analyzing code structure and control flow.
- Code Complexity Analysis:** Calculates various complexity metrics, including Cyclomatic Complexity, to highlight overly complex functions or methods that are difficult to understand, test, and maintain. This helps in refactoring efforts.
- Code Style & Best Practice Suggestions:** Provides recommendations for improving code style, adherence to coding standards (e.g., PEP 8 for Python), and best practices, promoting consistency across a codebase.
- Detailed Report Generation:** Generates comprehensive reports outlining detected issues, their severity, and suggested fixes, presented in an easily digestible format for developers.
- Customizable Rule Engine:** Allows users to define and configure custom rules for specific project requirements or coding standards, making the analyzer adaptable to various development environments.

- Implementation Highlights:**

- Abstract Syntax Tree (AST) Parsing:** Utilized Python's `ast` module to parse source code into an Abstract Syntax Tree. This allowed for a deep, structural analysis of the code, enabling the detection of logical errors and complex patterns that simple text-based analysis would miss.
- NLP for Code Pattern Recognition:** Employed NLP libraries like NLTK and spaCy to process code comments, variable names, and function names to identify semantic patterns. This helped in understanding the intent behind certain code blocks and detecting inconsistencies